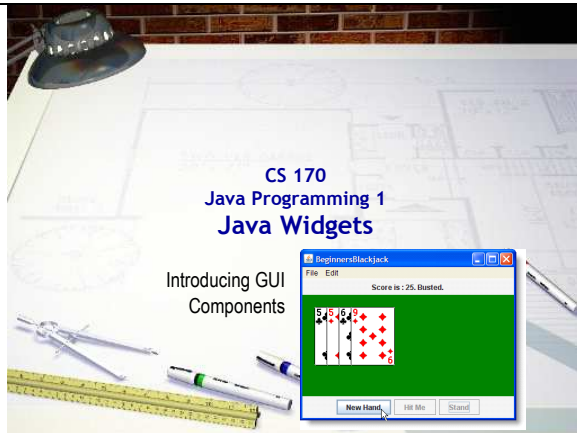




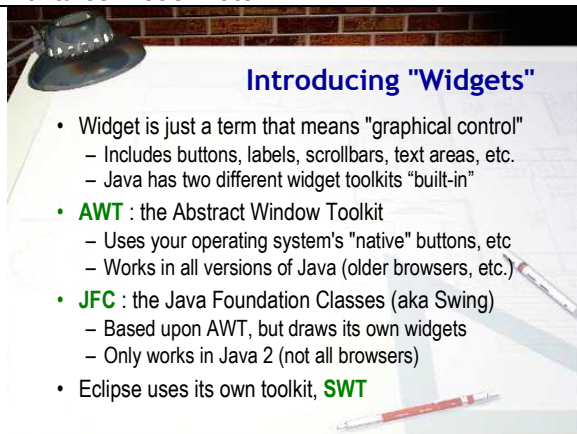
Slide 1
CS 170
Java Programming 1
Java Widgets
 Duration: 00:00:41
 Advance mode: Auto

Notes:

Hi folks. Welcome to the CS 170, Java Programming 1 lecture on Java Widgets.

While we've done graphical programming in this class (using the GImage and Graphics classes), you haven't yet worked with graphical "interactors" or GUI components, like buttons and labels. In this short introductory lecture, you'll learn how to add buttons and labels to your programs. That way the users of your video Blackjack program will be able to tell you whether they want to hit or stand, and you'll be able to tell them when they've won or lost.

Before we do that, though, let's take a look at that intriguing term, widget.



Slide 2
Introducing "Widgets"
 Duration: 00:03:32
 Advance mode: Auto

Notes:

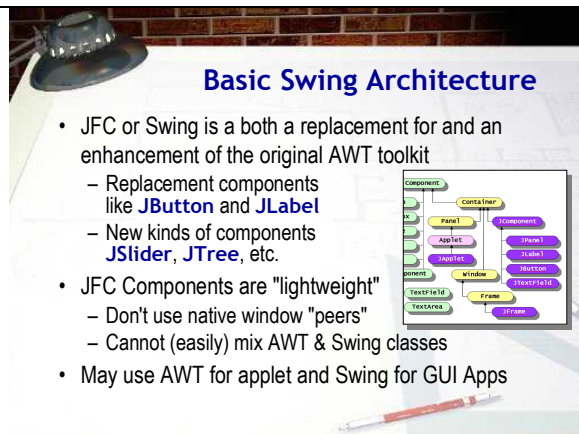
Widget is kind of a strange term, but in the programming world, it has a specific meaning. It refers to the graphical controls that are used to capture user input and to display your computer's output. This includes things like pushbuttons, checkboxes, radio buttons, labels, scrollbars, menus, text areas and so on.

No you've probably noticed that these widgets often look different on different platforms. The buttons and labels on a Mac look a lot different than those on Windows. Even on the same platform, though, the widgets can take on a different look and feel. In Windows XP, for instance, you can choose whether all of the controls will use the XP look, or the Windows Classic look. A collection of controls with a consistent behavior and (often) theme is called a widget "toolkit", and Java has two entirely different widget toolkits built into the Java class libraries.

The graphical widget toolkit that was introduced along with Java in 1996 was called the Abstract Window Toolkit or AWT. The AWT toolkit actually doesn't contain specific Java widgets, but instead uses your native operating system's toolkit. When you create a Java Button in your program, and then run it on your PC, the Java program "borrows" a Windows button from the operating system. When you run the same program on Mac OSX, the Java program borrows a Mac button instead. That means that the Java programs will generally take on the look and feel of the platform that they run on. It also means, though, that your program may look a lot different when run on a different computer, and sometimes you don't want that. Since the AWT is the older, original Java widget toolkit, AWT programs work in all versions of Java, even the oldest. So, if you're writing a GUI applet that just has to run on Internet Explorer 3 or Netscape Navigator 4, you have to use the AWT.

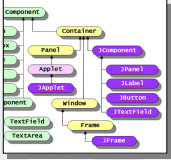
The JFC or Java Foundation Classes was introduced along with Java 2, around 2000. The JFC widgets are more commonly known by Sun's internal code name for the project that developed them: the Swing classes. The JFC widgets are based upon the AWT and still use some of the AWT classes, such as Color, but instead of using operating system widgets, like the AWT does, Swing widgets take complete responsibility for displaying themselves. That means that Swing programs can look exactly the same on each platform when they run. Because the Swing widgets weren't available in earlier versions of Java, though, programs that use them won't be able to run on older browsers.

While the Java Class Libraries include both the AWT and Swing toolkits, there's nothing to stop you from using an entirely different set of widgets. That's what Eclipse does; the folks at IBM who originally created Eclipse didn't especially like either Swing or the AWT and so they developed the Standard Widget Toolkit or SWT. While you can use the SWT in your programs, you'll have to distribute the SWT library along with your program classes, just like Eclipse does.



Basic Swing Architecture

- JFC or Swing is a both a replacement for and an enhancement of the original AWT toolkit
 - Replacement components like **JButton** and **JLabel**
 - New kinds of components **JSlider**, **JTree**, etc.
- JFC Components are "lightweight"
 - Don't use native window "peers"
 - Cannot (easily) mix AWT & Swing classes
- May use AWT for applet and Swing for GUI Apps



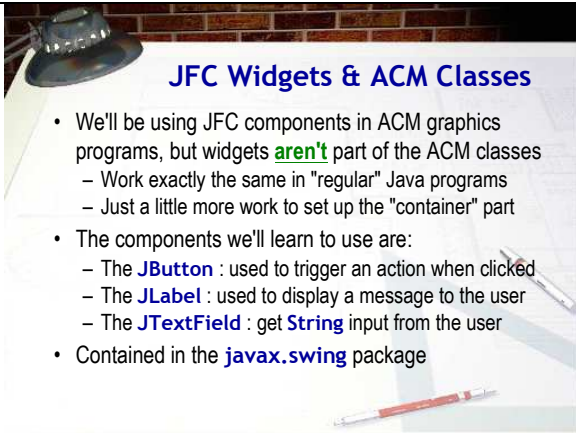
Slide 3
Basic Swing Architecture
Duration: 00:01:36
Advance mode: Auto

Notes:

In this class, we'll use the newer JFC or Swing classes instead of the AWT classes. The Swing classes include replacements for all of the components in the AWT. Where the AWT has the class Button, Swing has the class JButton; where the AWT has Label, Swing has JLabel and so on. Swing isn't merely a replacement library though; it's actually a much richer toolkit, that contains a whole raft of components not available in the AWT, with names like JSlider, JProgressBar, JTable, JTree and so on.

The Swing components are sometimes called "lightweight" components because they don't "borrow" buttons and labels from the native operating system, like the AWT does. (These borrowed buttons are sometimes called native window peers.) Because of this underlying difference in the way the widgets are implemented, though, you have to make a decision as to which one to use when writing your program. You can't decide to use all AWT widgets and then later add a JSlider to your program. You have to choose AWT or Swing; you can't mix them.

One thing that some people do, though, is to use the AWT classes for Java applets that need to run inside a browser. It's more likely if you post an applet on a Web page, that someone will access it with an older browser that can't display the newer Swing classes. For GUI applications, though, you'll probably want to use the newer, and more powerful, Swing classes.



JFC Widgets & ACM Classes

- We'll be using JFC components in ACM graphics programs, but widgets **aren't** part of the ACM classes
 - Work exactly the same in "regular" Java programs
 - Just a little more work to set up the "container" part
- The components we'll learn to use are:
 - The **JButton** : used to trigger an action when clicked
 - The **JLabel** : used to display a message to the user
 - The **TextField** : get **String** input from the user
- Contained in the **javax.swing** package

Slide 4

JFC Widgets & ACM Classes

Duration: 00:01:45

Advance mode: Auto

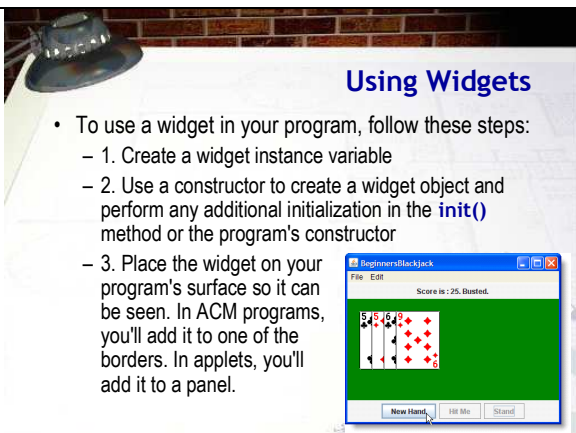
Notes:

In this class, we'll be using the JFC components inside our ACM graphics programs. The JFC components aren't part of the ACM libraries though, like the GImage and GRect classes are. The JFC components will work exactly the same way when you use them in a "regular" Java GUI program. The only difference is that with the ACM programs, it's a little easier to make the widgets look good as you add them to your program. In a regular Java GUI program, there's quite a bit of code required to set up the "container" part; code that the ACM GraphicsProgram class writes for you.

You can really write quite a few programs by learning to use just a few different components. At a minimum, you'll need controls for input, processing and output: the basic operations of every computer program. The components we'll use for that are:

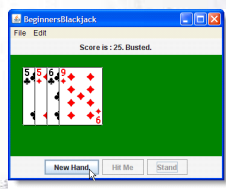
- The JButton widget, which we'll use to trigger the processing in your program. When someone clicks a button, we'll take a particular action.
- The JLabel widget, which we'll use to display the results of your processing back to the user.
- The JTextField widget, which will handle the input side of the equation.

To use any of these widgets in your program, you'll need to add a new import statement at the top of your code. Since these are all in the javax.swing package, just add import javax.swing.*; to the top of your file. The x in javax means that the classes are a standard extension to the base Java class libraries. Of course, since they're included in the Java Class libraries themselves, the word extension doesn't have much meaning.



Using Widgets

- To use a widget in your program, follow these steps:
 - Create a widget instance variable
 - Use a constructor to create a widget object and perform any additional initialization in the **init()** method or the program's constructor
 - Place the widget on your program's surface so it can be seen. In ACM programs, you'll add it to one of the borders. In applets, you'll add it to a panel.



Slide 5

Using Widgets

Duration: 00:01:10

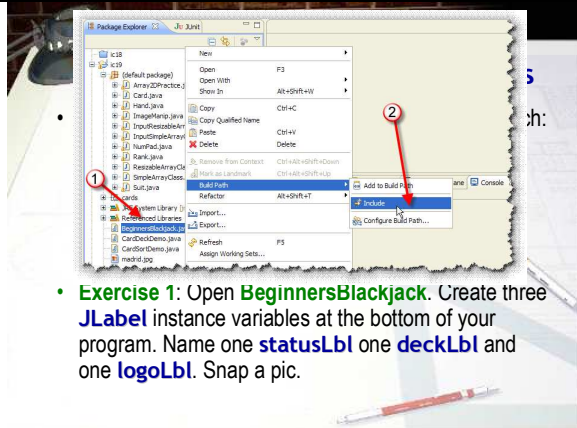
Advance mode: Auto

Notes:

To use any kind of widget in your program, whether it's a JButton, a JTextField or a JLabel, you'll always follow these three basic steps. Think of them as a checklist:

- First, you'll create a widget instance variable. You don't need to initialize it.
- Then, you'll use a widget constructor to create a widget object. Often, however, the widget constructor won't complete all of the required initialization. You may want to change the size or color of the widget. This additional initialization will go inside your program's constructor or, if you're creating an applet, the program's init() method.
- Finally, you need to place the widget on the surface of the program, so that the user can see and interact with it. In ACM programs, you'll do that by adding it to one of the borders of the program (that is, on the north or south) like the label at the top of this program, or the buttons on the

bottom. In an applet, you'd normally create a panel and then add the widget to that. As I mentioned earlier, though, the ACM method is a little easier.



Notes:

Now that you've seen the general pattern, let's apply it to the first, and one of the simplest, of the Swing core components, the JLabel. The JLabel class defines a set of objects that:

- can hold a single line of text, and/or an image
- know how to display themselves. (In other words, you don't have to use `drawString()` to print a message; the JLabel handles all of those details itself.
- can be positioned and moved around the screen.
- contains a surprising amount of information about their appearance, such as font size, text color, alignment, and so on.

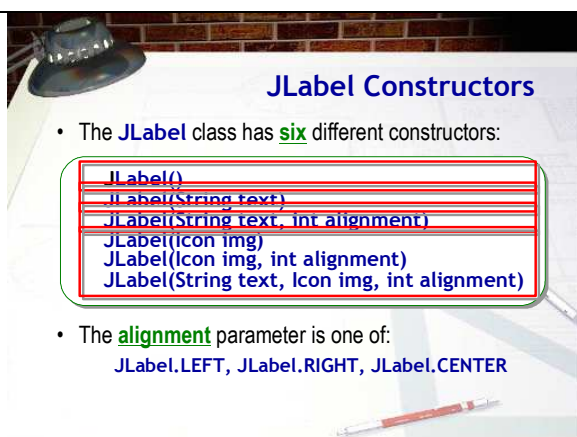
For our practical introduction (and for Exercise 1) we're going to work with the BeginnersBlackjack program. In the Eclipse Project Explorer, you'll find it at the bottom of the ic19 project files, underneath the "Referenced Libraries" icon. Right-click the program and choose Build Path->include from the context menu. Then, the file and scroll down to the bottom where you'll find the instance variables. Since you already know how to create instance variables, create three JLabel instance variables. Name one statusLbl, one deckLbl, and one logoLbl. Snap me a picture of your code.

Slide 6

The JLabel Class

Duration: 00:01:32

Advance mode: Auto



Notes:

The Label class has six different constructors as shown here.

Use the first constructor, the one that takes no arguments, if you intend to modify the message displayed as your program runs, but don't have a valid message when the program first starts up. If your Label object is going to display the results of a calculation, for instance, you might not want to display anything until the calculation is complete.

The second constructor is the most common for simple text labels. Simply provide a message when you construct your JLabel, and the object itself will take care of the painting chores. The third constructor allows you to change the alignment of the message within the JLabel object itself. You do that by passing a second argument, which can be one of

Slide 7

JLabel Constructors

Duration: 00:01:50

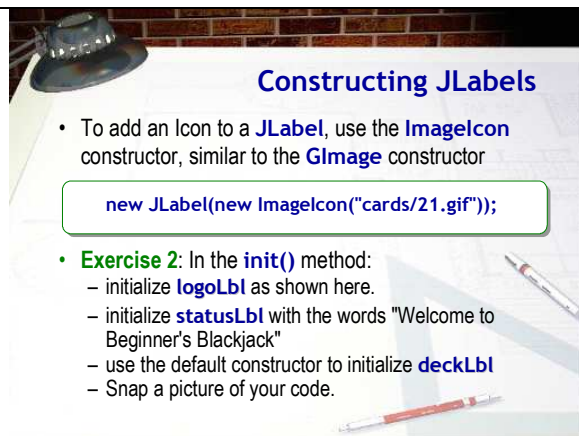
Advance mode: Auto

these class constants:

* JLabel.LEFT, JLabel.CENTER, JLabel.RIGHT

Two words of warning with this constructor, though. First, be careful with the spelling of these constants. Note that the class name is proper-case, while the constant name is upper-case. You must spell the constant correctly for this to work. Second, students are frequently frustrated when trying to use the alignment constants because they don't do exactly what you'd expect. Specifying JLabel.CENTER, for instance, does not center the label on the applet; instead, it centers the text inside the label. Since the Label objects normally size themselves according to the text they display, the effect is very hard to see. If you make the label very wide, however, it becomes apparent.

The last three constructors allow you to use images, either alone or with text. To do that, you'll use the ImageIcon class. Let's take a look at that next.



Constructing JLabels

- To add an Icon to a **JLabel**, use the **ImageIcon** constructor, similar to the **Image** constructor

```
new JLabel(new ImageIcon("cards/21.gif"));
```

- Exercise 2:** In the **init()** method:
 - initialize **logoLbl** as shown here.
 - initialize **statusLbl** with the words "Welcome to Beginner's Blackjack"
 - use the default constructor to initialize **deckLbl**
 - Snap a picture of your code.

Slide 8

Constructing JLabels

Duration: 00:01:19

Advance mode: Auto

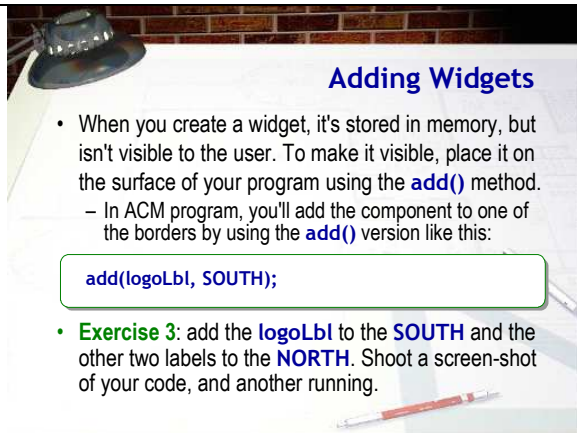
Notes:

The ImageIcon constructor is almost identical to the Image constructor. For instance, if you want to display the cards image 21.gif, you'd use code like this. This creates an ImageIcon using the constructor, and then passes that to the fourth JLabel constructor.

OK, let's put this to work. For Exercise 2, locate the init() method (immediately before the section that creates the instance variables), and then locate the comment that says // EXERCISE: Initialize the three JLabels. Below the comment:

- initialize the variable logoLbl using the code shown on this slide.
- initialize the JLabel variable named statusLbl with the words "Welcome to Beginner's Blackjack"
- and use the default or no-parameter constructor to initialize the variable named deckLbl.

When you're done, shoot me a screenshot of your code, and paste it into your IC document. Your code will probably compile, but you won't see anything if you run it, because we haven't yet added the label to the surface of your applet.



Adding Widgets

- When you create a widget, it's stored in memory, but isn't visible to the user. To make it visible, place it on the surface of your program using the `add()` method.
 - In ACM program, you'll add the component to one of the borders by using the `add()` version like this:

```
add(logoLbl, SOUTH);
```

- Exercise 3:** add the `logoLbl` to the `SOUTH` and the other two labels to the `NORTH`. Shoot a screen-shot of your code, and another running.

Slide 9 

Adding Widgets

Duration: 00:01:31

Advance mode: Auto

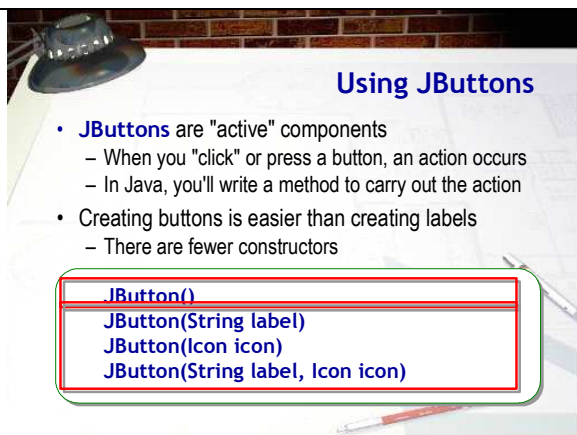
Notes:

When you create `JLabel` or a `JButton` object, it's constructed and stored in memory, but it isn't yet visible to the user of your program. To make a GUI component visible, you need to add it to some kind of container. In this case, that's your `ACM GraphicsProgram` itself. To add a component to a container, you use, naturally, the method named `add()`. (This is true in all GUI programs, not just ACM programs).

In ACM programs, though, you also have to tell the component which border location you want to place it into, using the constants `NORTH`, `SOUTH`, `EAST` or `WEST`. Note when using these that they are all upper-case.

That sounds easy, right? OK, let's try it out. For Exercise 3, add the `logoLbl` to the `SOUTH` and the other two labels to the `NORTH`. You'll need one statement for each `JLabel`, so you'll have two `add()` statements with the word `NORTH`. Once you've done that, save and then run your program. You should see two of the labels appear on the surface of your applet. One of them, `deckLbl`, doesn't appear since it doesn't contain either an image or any text.

Shoot me a screen-shot and paste it into your exercise document. Then, underneath that, paste the portion of code you used to add the `JLabels` to the screen.



Using JButtons

- JButtons** are "active" components
 - When you "click" or press a button, an action occurs
 - In Java, you'll write a method to carry out the action
- Creating buttons is easier than creating labels
 - There are fewer constructors

```
JButton()
JButton(String label)
JButton(Icon icon)
JButton(String label, Icon icon)
```

Slide 10 

Using JButtons

Duration: 00:01:03

Advance mode: Auto

Notes:

Now, let's turn our attention to a second component. Let's face it, the `JLabel` really doesn't know a whole lot of tricks. The `JButton`'s middle name, by contrast, is action. If you want something to happen in your Java program, use a `JButton` object to start it off. Then, you'll use a method to carry out the action.

Creating buttons is a little easier than creating labels, since there is no concept of "alignment". That means you only have to learn four constructors, instead of six. There is a default or no-parameter `JButton` constructor, but this isn't as useful as the default `JLabel` constructor. You rarely want to create such an "empty" button. The other three constructors allow you to specify the text, icon, or combination of text and icon that will appear on the button. You use the `ImageIcon` class to create an icon for a `JButton`, just as you did for the `JLabel` class.



Express Yourself

- **Exercise 4:** follow the same pattern you used with **JLabel** to add some **JBUTTONS** to your program
- Add three **JButton** instance variables
 - **newHandBtn**: deal a new hand
 - **hitBtn**: deal a new card to the player's hand
 - **standBtn**: see the dealer's cards and score game
- Construct all three buttons in **init()**
- Add the buttons to the south border with the logo
- Shoot a screen-shot of your code, and another shot of your program running.

Slide 11 

Express Yourself

Duration: 00:01:27

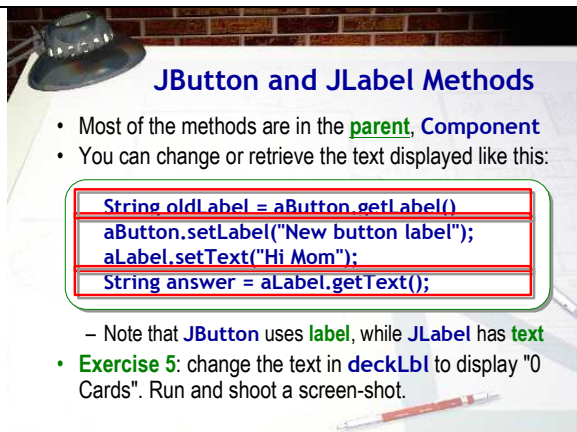
Advance mode: Auto

Notes:

Since JButtons are actually simpler than JLabels, let's see if you can do all three steps in the "widget pattern" by yourself. Here's what I want you to do for Exercise 4:

- First, add three JButton instance variables (not objects) below the place where you created the JLabel instance variables. The newHandBtn will deal a new hand. The hitBtn will deal a new card to the player's hand, while the standBtn will be the signal for the dealer to display his or her cards and then score the game. (I realize that this isn't exactly how Blackjack works, but that's why this is called Beginner's blackjack.).
- Next, use the appropriate text constructor to initialize all three of your variables. Place this code in the init() method, immediately following the code that initialized the JLabel instance variables. Use the text constructor and appropriate verbiage on the face of each button.
- Finally, add the buttons to the SOUTH portion of your program. They'll share the region below the table with the logoLbl you placed there earlier.

When you're done, run your program again and give me another screenshot. Then, paste the code for your program (or a screenshot of the code) below the first.



JButton and JLabel Methods

- Most of the methods are in the **parent, Component**
- You can change or retrieve the text displayed like this:

```
String oldLabel = aButton.getLabel();
aButton.setLabel("New button label");
aLabel.setText("Hi Mom");
String answer = aLabel.getText();
```

- Note that **JButton** uses **label**, while **JLabel** has **text**
- **Exercise 5:** change the text in **deckLbl** to display "0 Cards". Run and shoot a screen-shot.

Slide 12 

JButton and JLabel Methods

Duration: 00:01:34

Advance mode: Auto

Notes:

Like other objects, JButtons and JLabels have many methods you can use to modify their behavior and appearance. Most of these methods aren't in the JButton or JLabel classes themselves, but in a common ancestor class named Component. Such sharing of common code is one of the benefits of the object-oriented technique of inheritance, which you'll study in the next few lessons.

To finish up this lecture, though, let's look at the methods you can use to change the text that appears on a JLabel or a JButton after the widget has already been created. You might want to change the text that appears on particular buttons in response to events in the game, for instance.

To change the text that appears on a widget, you use a set() method. For the Label class, the method is called setText and for the button class, the method is setLabel. (It seems like it should be the other way around, doesn't it.) To retrieve the existing text, you'll use one of the get() methods: getLabel() for the JButton and getText() for the JLabel.

For your last Exercise in this lesson, add code in your init() method to change the text displayed by the deckLbl variable. (That's the one that you created originally empty, remember?) Have it display the Phrase "0 Cards". Run, shoot a screenshot and paste it into your IC 19 document.

